

1. Two-sum using HashMap.

Question: Given an array of integers (nums) and an integer target, find the indices of the two numbers in the array that add up to the target.

Simple Explanation: We use a **HashMap** to solve this very fast. We iterate through the array once, and for each number, we calculate the complement needed (target minus the current number). We check if this required complement already exists in our map. If it does, we found the pair; if not, we add the current number and its index to the map for future checks.

```
import java.util.HashMap;
import java.util.Map;

public class TwoSumFinder {
    /**
     * @param nums The array of numbers to search.
     * @param target The target sum.
     * @return The indices of the two numbers.
     */
    public static int[] findTwoSum(int[] nums, int target) {
        Map<Integer, Integer> numMap = new HashMap<>(); // Key: number,
Value: index

        for (int i = 0; i < nums.length; i++) {
            int complement = target - nums[i];

            // Check if the required complement is already in the map
            if (numMap.containsKey(complement)) {
                // Pair found: return the complement's index and the
current index
                return new int[]{numMap.get(complement), i};
            }

            // Store the current number and its index for future checks
            numMap.put(nums[i], i);
        }

        // If no pair is found (based on the problem statement, this
should not be reached)
        throw new IllegalArgumentException("No two sum solution found for
the given target.");
    }
}
```

2. Check for anagram using HashMap.

Question: Write a function to determine if two given strings (s1 and s2) are anagrams of each other. Anagrams must have the same characters with the same frequencies, regardless of order.

Simple Explanation: We check for anagrams by using a **HashMap** to track character counts. We loop through the first string (s1), increasing the count for every character. Then, we loop through the second string (s2), decreasing the counts. If the final map contains all zeros, or if we try to decrease a count that's already zero, they are not anagrams.

```
import java.util.HashMap;
import java.util.Map;

public class AnagramChecker {
    /**
     * @param s1 The first string.
     * @param s2 The second string.
     * @return true if s1 and s2 are anagrams, false otherwise.
     */
    public static boolean areAnagrams(String s1, String s2) {
        if (s1 == null || s2 == null || s1.length() != s2.length()) {
            return false; // Must be same length and non-null
        }

        // Map to store character counts from s1
        Map<Character, Integer> charCounts = new HashMap<>();

        // 1. Count characters in the first string (s1)
        for (char c : s1.toLowerCase().toCharArray()) {
            charCounts.put(c, charCounts.getOrDefault(c, 0) + 1);
        }

        // 2. Decrement counts based on the second string (s2)
        for (char c : s2.toLowerCase().toCharArray()) {
            // Check if character is missing or count is already zero
            if (!charCounts.containsKey(c) || charCounts.get(c) == 0) {
                return false;
            }
            charCounts.put(c, charCounts.get(c) - 1);
        }

        // 3. Final check: ensure the map is "empty" (all counts are 0).
        // Since we checked the length initially, checking if the map
        // contains all zeros is sufficient.
        return charCounts.values().stream().allMatch(c -> c == 0);
    }
}
```

3. Check if two arrays are equal using HashMap.

Question: Determine if two integer arrays (arr1 and arr2) are equal, meaning they contain the exact same elements with the exact same frequencies, regardless of the order of elements.

Simple Explanation: To check for equality regardless of order, we use a **HashMap** to count the frequency of elements in the first array (arr1). We then loop through the second array (arr2), decreasing the counts. If any count drops below zero, or if the initial lengths are different, they are not equal.

```
import java.util.HashMap;
import java.util.Map;

public class ArrayEqualityChecker {
    /**
     * @param arr1 The first array.
     * @param arr2 The second array.
     * @return true if both arrays contain the same elements with the same
     frequency.
     */
    public static boolean areEqual(int[] arr1, int[] arr2) {
        if (arr1 == null || arr2 == null || arr1.length != arr2.length)
        {
            return false;
        }

        Map<Integer, Integer> counts = new HashMap<>();

        // 1. Count elements in the first array
        for (int num : arr1) {
            counts.put(num, counts.getOrDefault(num, 0) + 1);
        }

        // 2. Decrement counts based on the second array
        for (int num : arr2) {
            // If num is not in the map, or its count is already 0, the
            arrays are unequal.
            if (!counts.containsKey(num) || counts.get(num) == 0) {
                return false;
            }
            counts.put(num, counts.get(num) - 1);
        }

        // 3. Final check: Since lengths are equal, if all counts are now
        0, they are equal.
        // We use stream().allMatch() to verify all remaining values are
        zero.
        return counts.values().stream().allMatch(c -> c == 0);
    }
}
```

4. Find common elements between two lists using Set.

Question: Given two lists of integers (list1 and list2), find and return a list containing only the elements that are present in both input lists.

Simple Explanation: We use a **HashSet** to store all elements from the first list. This allows for extremely fast lookups ($O(1)$). We then loop through the second list, checking if the element exists in the set. If it does, we add it to our result list and remove it from the set to avoid adding duplicates if the second list contains them.

```
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

public class CommonElementsFinder {
    /**
     * @param list1 The first list.
     * @param list2 The second list.
     * @return A list containing only elements common to both input lists.
     */
    public static List<Integer> findCommonElements(List<Integer> list1,
List<Integer> list2) {
        Set<Integer> set1 = new HashSet<>(list1); // Use Set for fast O(1)
lookups
        List<Integer> commonElements = new ArrayList<>();

        // Check if elements from list2 are present in the set
        for (int num : list2) {
            if (set1.contains(num)) {
                commonElements.add(num);
                set1.remove(num); // Remove from the set to ensure unique
elements in the result list
            }
        }
        return commonElements;
    }
}
```